机器学习基础·讲义

梯度下降：从零到收敛的完整推导

目录

1 把“学习”翻译成“最小化” 2

2 一维：导数的符号告诉你往哪走 3

3 多维：从偏导到梯度 6

4 严格化：下降引理 9

5 二次函数的完整解 12

6 三种梯度下降 13

7 手算全过程 15

8 收敛性到底保证了什么 16

9 三个真实模型的推导 18

10 八个常见误区 20

11 自测 22

12 代码附录 24

1 把“学习”翻译成“最小化”

1.1 三个东西：参数、损失、目标

训练一个模型，说到底只做一件事：**找一组参数，让某个衡量“错得多离谱”的数尽可能小**。这句话里其实有三个对象，先把它们分清楚，后面的推导才有落脚点。

对象	记号	含义	在直线拟合里是什么
参数	$\mathbf{w} \in \mathbb{R}^d$	模型里可以被调整的数，d 是它的个数	斜率，若带截距则还有 b
损失函数	$L(\mathbf{w})$	给定一组参数时，模型错得有多严重；越小越好	所有点的预测偏差平方和
目标	$\mathbf{w}^\star = \arg\min_{\mathbf{w}} L(\mathbf{w})$	让损失取最小的那组参数	最小二乘解

注意 L 是一个**从 $\mathbb{R}^d$ 到 $\mathbb{R}$ 的函数**：你喂给它 d 个数，它还你一个数。梯度下降要做的，是在这个 d 维的“地形”上，从一个初始点出发，一步步走到谷底。

> **先建立一个图像**
>
> 把 $L(\mathbf{w})$ 想成一片山地，$\mathbf{w}$ 是你站在地图上的坐标（经度、纬度……共 d 个方向），L 是你脚下的海拔。你的目标是走到最低的谷底。但你是个**蒙着眼睛的人**：看不见全局，只能用脚感受脚下这一小块的坡度。梯度下降就是“摸一摸脚下往哪边最陡，就往反方向迈一步”的规则。
>
> 这个比喻会把后面所有结论都解释清楚：为什么只能保证局部、为什么步长太大会摔下悬崖、为什么地形狭长时会来回横跳。

1.2 一个能算到底的例子

抽象的话说到这里，先看一个能手算的例子，它在第 7 节会被完整走一遍。

假设有三个观测点 $(x, y) = (1,\, 2.1),\ (2,\, 3.9),\ (3,\, 6.2)$，我们想用一条过原点的直线 $\hat{y} = wx$ 去拟合。参数只有一个：斜率 w。

预测偏差是 $\hat{y}_i - y_i = wx_i - y_i$，把三个偏差的平方加起来（平方是为了让正负偏差都算“错”，且对大偏差惩罚更重），再除以 2 只是为了后面求导时消掉系数：

$$L(w) = \frac{1}{2}\sum_{i=1}^{3}(wx_i - y_i)^2 = \frac{1}{2}\Big[(w-2.1)^2 + (2w-3.9)^2 + (3w-6.2)^2\Big]$$

展开合并同类项（你可以自己验一遍）：

$$L(w) = 7w^2 - 28.5w + 29.03$$

这是一条开口向上的抛物线。它的最小值在哪？对 w 求导令其为零：

$$L'(w) = 14w - 28.5 = 0 \quad \Longrightarrow \quad w^\star = \frac{28.5}{14} \approx 2.036$$

答案精确可得，因为这是一维二次函数。但真实模型里 L 可能是几亿个参数的复杂函数，**没有公式解**，只能一步步逼近。梯度下降就是那个“一步步逼近”的算法。

> **这一节要记住的**
> 梯度下降不是“另一种求最小值的方法”，而是在**没有解析解**时唯一可行的通用手段。它的全部信息来自“当前点的局部斜率”，所以它的所有优点和缺陷，都源于这个信息限制。

2 一维：导数的符号告诉你往哪走

2.1 导数究竟说了什么

一元函数 $f(w)$ 在 w_0 处的导数定义为

$$f'(w_0) = \lim_{h \to 0} \frac{f(w_0 + h) - f(w_0)}{h}$$

这个式子的意思是：**在 w_0 处把 w 增大一点点，函数值会以多大的速率变化**。它是一个“变化率”，不是一个“位置”。

由此直接得到两条判据，这是整篇文章的地基：

- $f'(w_0) > 0$：往右走，函数**上升**。所以想下降，就该往**左**走，即减小 w。
- $f'(w_0) < 0$：往右走，函数**下降**。所以想下降，就该往**右**走，即增大 w。

两种情况能不能写成一个式子？可以。取

$$w_1 = w_0 - \eta f'(w_0), \qquad \eta > 0$$

当 $f'(w_0) > 0$ 时，减去一个正数，w 变小，往左走，正确；当 $f'(w_0) < 0$ 时，减去一个负数等于加上一个正数，w 变大，往右走，也正确。**负号一次性处理了两种情形，**

这就是“减”的来历，它不是什么深奥的规定，只是把“顺着坡往下”这件事写成了一个统一公式。

2.2 为什么“减一点”就一定会下降

上面的论证还是定性的。要定量知道“走多远、降多少”，需要泰勒展开。

若 f 在 w_0 处可导，则

$$f(w_0 + h) = f(w_0) + f'(w_0)\,h + o(h)$$

其中 $o(h)$ 表示“比 h 更快趋于零的项”，即 $\lim_{h\to 0} o(h)/h = 0$。这句话的含义是：**当 h 足够小时，函数的变化几乎完全由线性项 $f'(w_0)h$ 决定**，剩下的部分是更高阶的小量。

代入 $h = -\eta f'(w_0)$：

$$f\big(w_0 - \eta f'(w_0)\big) = f(w_0) - \eta\big[f'(w_0)\big]^2 + o(\eta)$$

关键在 $-\eta[f'(w_0)]^2$ 这一项：只要 $f'(w_0) \neq 0$ 且 $\eta > 0$，它就**严格为负**。而 $o(\eta)$ 在 η 足够小时可以任意小，小到无法抵消这一项。于是存在某个 $\delta > 0$，使得所有 $0 < \eta < \delta$ 都满足

$$f\big(w_0 - \eta f'(w_0)\big) < f(w_0)$$

> **严格性的来源**
>
> 上面这一步不是“近似”，而是严格结论。它的逻辑是：*极限为正* $\Longrightarrow$ *存在邻域使函数值为正*。把导数定义写成
>
> 取 $h = -\eta f'(w_0)$，则当 $\eta \to 0^+$ 时，比值的极限是 $-f'(w_0)^2 < 0$。按极限定义，存在 δ，使 $0 < \eta < \delta$ 时该比值为负，即分子 $f(w_0+h) - f(w_0) < 0$。这就证明了“小步必然下降”。
>
> 注意结论里的“存在 δ”：**下降只在足够小的步长下成立**，这个限定会在第 4 节变成具体的数值上界。

2.3 步长的三种命运

η 叫学习率（learning rate）或步长。它决定了沿着斜率方向迈多远。用同一个抛物线 $L(w) = 7w^2 - 28.5w + 29.03$（$L'' = 14$）从 $w_0 = 0$ 出发，可以清楚看到三种结局：

η	前几步的 w	现象	原因
0.02	$0.57 \to 0.98 \to 1.28 \to 1.49 \to \cdots$	稳稳逼近 2.036，但很慢	每步只走一点点，误差按固定比例缩小
0.12	$3.42 \to 1.09 \to 2.68 \to 1.60 \to \cdots$	来回摆动，摆幅逐渐变小	一步迈过了谷底，但还没越过对面的坡
0.15	$4.28 \to -0.43 \to 4.75 \to -0.95 \to \cdots$	越摆越大，发散	每步都跨到比出发点更高的位置，能量越积越多

初值 $w_0 = 0$，目标 $w^\star = 2.036$。$\eta = 0.143$ 是临界点，恰好 $2/L''$。

为什么临界点正好是 $2/14 \approx 0.143$？这个数字不是巧合。对二次函数，我们可以把每步的误差写成一个递推。

令 $e_k = w_k - w^\star$ 表示“离谷底还有多远”。因为 $L'(w) = 14(w - w^\star) = 14e_k$，更新式变成

$$e_{k+1} = w_{k+1} - w^\star = w_k - \eta \cdot 14e_k - w^\star = (1 - 14\eta)\, e_k$$

于是 $e_k = (1 - 14\eta)^k e_0$。要让误差趋于零，必须

$$|1 - 14\eta| < 1 \quad \Longleftrightarrow \quad 0 < \eta < \frac{2}{14} \approx 0.143$$

- $\eta < 1/14$：$0 < 1 - 14\eta < 1$，误差单调缩小，**不摆动**；
- $1/14 < \eta < 2/14$：$-1 < 1 - 14\eta < 0$，误差缩小但每步变号，**来回摆**；
- $\eta > 2/14$：$|1 - 14\eta| > 1$，误差被放大，**发散**。

这三条正好对应表里的三行。**“步长必须小于 $2/$曲率”是梯度下降最基本的一条纪律**，第 4 节会把它推广到多维。

> **一个容易误解的点**
>
> 「学习率越小越安全」只对了一半。η 太小确实不会发散，但收敛会慢到不可接受；而且在实际训练中，过小的步长会让参数卡在梯度接近零的平坦区域，误以为已经收敛。η 的选择是**稳定性与速度的权衡**，不是单调的“越小越好”。

3 多维：从偏导到梯度

3.1 偏导：把其他变量按住不动

模型几乎不会只有一个参数。以 $L(w_1, w_2)$ 为例，它是一个曲面。站在某一点，脚下有无数个方向可以走，每个方向的坡度都可能不同。

先看最简单的一类方向：**沿着某根坐标轴走**。只让 w_1 变、w_2 不动，此时函数对 w_1 的变化率叫偏导数：

$$\frac{\partial L}{\partial w_1} = \lim_{h \to 0} \frac{L(w_1 + h,\ w_2) - L(w_1, w_2)}{h}$$

它的计算方法和一元导数完全一样：**把 w_2 当成常数**。例如

$$L(w_1, w_2) = \frac{1}{2}\left(w_1^2 + 3w_2^2\right) \quad \Longrightarrow \quad \frac{\partial L}{\partial w_1} = w_1, \qquad \frac{\partial L}{\partial w_2} = 3w_2$$

偏导告诉你“只动这一个参数时，往哪边、以多快的速率变化”。但它**不能直接拿来更新**：如果分别用 $-\eta \partial L/\partial w_1$ 和 $-\eta \partial L/\partial w_2$ 去改两个参数，你其实是在同时沿两个坐标轴走，而“同时走”的合成方向是不是最陡的？这需要先回答一个更基本的问题。

3.2 方向导数：任意方向的坡度

设 $\mathbf{d}$ 是一个单位向量（$\|\mathbf{d}\| = 1$），表示一个方向。从点 $\mathbf{w}$ 沿 $\mathbf{d}$ 走，函数的变化率叫**方向导数**：

$$D_{\mathbf{d}}L(\mathbf{w}) = \lim_{t \to 0^+} \frac{L(\mathbf{w} + t\mathbf{d}) - L(\mathbf{w})}{t}$$

由多元链式法则（把 $L(\mathbf{w} + t\mathbf{d})$ 看成 t 的一元函数，对 t 求导）：

$$\frac{d}{dt} L(\mathbf{w} + t\mathbf{d})\Big|_{t=0} = \sum_{j=1}^{d} \frac{\partial L}{\partial w_j} \cdot \frac{d(w_j + t d_j)}{dt} = \sum_{j=1}^{d} \frac{\partial L}{\partial w_j} d_j = \nabla L(\mathbf{w})^\top \mathbf{d}$$

右边出现的这个向量，就是**梯度**：把所有偏导按顺序排成一列。

$$\nabla L(\mathbf{w}) = \begin{pmatrix} \dfrac{\partial L}{\partial w_1} \\ \vdots \\ \dfrac{\partial L}{\partial w_d} \end{pmatrix}$$

> **这个恒等式是全部推导的枢纽**
>
> $$D_{\mathbf{d}}L(\mathbf{w}) = \nabla L(\mathbf{w})^\top \mathbf{d}$$
>
> 它说的是：**任意方向的坡度，等于梯度与该方向的内积**。左边是“几何”（沿某个方向走有多陡），右边是“代数”（两个向量的夹角余弦乘以长度）。整个梯度下降的理论，就是反复使用这个等式。

3.3 哪个方向最陡

现在可以回答“往哪走下降最快”了。要最小化 $D_{\mathbf{d}}L = \nabla L^\top \mathbf{d}$，约束是 $\|\mathbf{d}\| = 1$。用 Cauchy–Schwarz 不等式：

$$\nabla L^\top \mathbf{d} \;\geq\; -\|\nabla L\|\,\|\mathbf{d}\| \;=\; -\|\nabla L\|$$

等号成立的条件是 $\mathbf{d}$ 与 ∇L **反向且共线**，即 $\mathbf{d} = -\nabla L / \|\nabla L\|$。代回去验证：

$$\nabla L^\top \left(-\frac{\nabla L}{\|\nabla L\|} \right) = -\frac{\|\nabla L\|^2}{\|\nabla L\|} = -\|\nabla L\|$$

所以：**负梯度方向是单位步长下函数下降最快的方向**，“最速下降法”（steepest descent）这个名字就是这么来的。

> **”最快”的准确含义**
>
> 这里的最快是在**无穷小步长**意义下、并且用**欧氏长度**衡量“步长”时成立的。换成有限步长，“负梯度方向下降最多”就可能不成立，第 5 节的二次函数例子会给出反例。换成别的长度定义（比如每个坐标单独限幅），最速方向也不再是负梯度。这个限定非常关键，第 10 节会专门讲。

3.4 为什么“负梯度”能同时照顾所有参数

上面的证明是代数的，但直觉上“为什么偏偏是它”值得单独说。把一阶变化按参数拆开：

$$\Delta L \approx \sum_{j=1}^{d} \frac{\partial L}{\partial w_j} \Delta w_j$$

每一项是“第 j 个参数这一步带来的变化”。现在取 $\Delta w_j = -\eta\, \partial L/\partial w_j$，每一项变成

$$\frac{\partial L}{\partial w_j} \Delta w_j = -\eta \left(\frac{\partial L}{\partial w_j} \right)^2 \leq 0$$

每一项都非正，没有任何一个参数在往上拉。这是负梯度方向独有的性质。换任何别的方向，必然有某些 Δw_j 与 $\partial L/\partial w_j$ 同号，那些参数就在“帮倒忙”，需要靠其他参数的下降来补偿，净效果只会更差。

数值验证

取 $L = \frac{1}{2}\|\mathbf{w}\|^2$，点 $\mathbf{w} = (1, 3)$，则 $\nabla L = (1, 3)$，$\|\nabla L\| = \sqrt{10} \approx 3.162$。固定步长 $\|\Delta \mathbf{w}\| = 0.5$，把方向从 $-\nabla L$ 开始逐渐旋转，观察两项贡献：

与 $-\nabla L$ 夹角	$\Delta \mathbf{w}$	w_1 项	w_2 项	净 ΔL
0°	$(-0.158, -0.474)$	-0.158	-1.423	**-1.456**
45°	$(-0.447, -0.224)$	-0.447	-0.671	-0.993
90°	$(-0.474, +0.158)$	-0.474	$+0.474$	$+0.125$
135°	$(-0.224, +0.447)$	-0.224	$+1.342$	$+1.243$
180°	$(+0.158, +0.474)$	$+0.158$	$+1.423$	$+1.706$

第一行两项都是负的，净下降最多；角度一大，w_2 那项翻正并抵消 w_1 的收益；90° 时恰好抵消（一阶为零，只剩二阶项 $+0.125$）。

3.5 梯度与等高线垂直

等高线是“函数值相同的点的集合”。沿等高线的切线方向 $\mathbf{d}$ 走，函数值不变，所以 $D_{\mathbf{d}}L = \nabla L^{\top}\mathbf{d} = 0$。这说明**梯度与等高线的切线正交**。

把两件事合起来看：梯度垂直于等高线、指向函数值增大的那一侧，负梯度则垂直指向减小的一侧。于是“梯度下降”在地图上就是**每一步都垂直穿过等高线，朝更低的那条等高线走**。这也解释了为什么在狭长的山谷里它会左右横跳：垂直穿过等高线的方向，恰好是山谷的横向，而不是沿着谷底延伸的方向。

> **一个小结**
>
> 到这里，更新公式的每一个零件都有了理由：
> $\mathbf{w}_{k+1} = \mathbf{w}_k - \eta\nabla L(\mathbf{w}_k)$
> **减号**：让方向与梯度相反（3.3 的 Cauchy–Schwarz）；
> **梯度**：唯一让所有参数同向下坡的方向（3.4）；
> η：控制走多远，必须小到一阶近似仍然有效（2.3、4.3）。

4 严格化：下降引理

4.1 我们到底需要什么假设

第 2 节用 $o(h)$ 说明了“足够小的步长会下降”，但“足够小”是多大？在多维里怎么判断？要给出可用的结论，需要对函数的光滑程度加一个量化假设。

定义（β-光滑）：若 L 可微，且存在常数 $\beta > 0$ 使得对任意 $\mathbf{w}, \mathbf{v}$ 都有

$$\|\nabla L(\mathbf{w}) - \nabla L(\mathbf{v})\| \le \beta\|\mathbf{w} - \mathbf{v}\|$$

则称 L 是 β-光滑的。直观上，它说**梯度不会剧烈变化**，也就是曲面不会突然折起来；β 是“曲率的上界”。对二次函数 $L = \frac{1}{2}\mathbf{w}^{\top}\mathbf{A}\mathbf{w}$，$\nabla L = \mathbf{A}\mathbf{w}$，可以验证 $\beta = \lambda_{\max}(\mathbf{A})$，即最大特征值。

β-光滑有一个极其好用的等价形式（对凸函数它是等价的，一般情形下由积分可得）：

$$L(\mathbf{w} + \Delta) \;\le\; L(\mathbf{w}) + \nabla L(\mathbf{w})^{\top}\Delta + \frac{\beta}{2}\|\Delta\|^2 \tag{4.1}$$

右边是一个**二次上界**：真正的函数值，永远不超过“切线 + 一个抛物线”。这个不等式就是泰勒展开加上余项控制，把第 2 节的 $o(h)$ 变成了具体的 $\frac{\beta}{2}\|\Delta\|^2$。

4.2 引理与证明

> **下降引理**
> 设 L 是 β-光滑的，取 $\Delta = -\eta\nabla L(\mathbf{w})$，则

证明：把 $\Delta = -\eta\nabla L$ 代入 (4.1)，记 $\mathbf{g} = \nabla L(\mathbf{w})$：

$$\begin{aligned} L(\mathbf{w} - \eta\mathbf{g}) &\le L(\mathbf{w}) + \mathbf{g}^\top(-\eta\mathbf{g}) + \frac{\beta}{2}\| - \eta\mathbf{g}\|^2 \\ &= L(\mathbf{w}) - \eta\|\mathbf{g}\|^2 + \frac{\beta\eta^2}{2}\|\mathbf{g}\|^2 \\ &= L(\mathbf{w}) - \eta\left(1 - \frac{\beta\eta}{2}\right)\|\mathbf{g}\|^2 \end{aligned}$$

证毕。□

4.3 步长上界的来源

看括号里的 $1 - \frac{\beta\eta}{2}$。只要它为正，即

$$\eta < \frac{2}{\beta}$$

右边就是严格下降，而且下降量与 $\eta\|\mathbf{g}\|^2$ 成正比。这一个不等式解释了前面所有的现象：

- **为什么 η 太大会发散**：$\eta > 2/\beta$ 时系数为负，L 反而增大；且每步放大，误差像等比数列一样爆掉。
- **为什么一维的临界值是 $2/L''$**：一维时 $\beta = |L''|$，完全吻合第 2 节算出的 $2/14$。
- **为什么特征尺度重要**：β 是最大曲率；把所有特征放大 10 倍，β 放大 100 倍，允许的步长就缩小 100 倍。这就是不标准化时训练容易崩的直接原因。
- **为什么取 $\eta = 1/\beta$ 是“标准答案”**：此时系数 $1 - \frac{1}{2} = \frac{1}{2}$，每步至少下降 $\frac{1}{2\beta}\|\mathbf{g}\|^2$，是保证下降的前提下能取的最大步长。

但它只是充分条件

$\eta < 2/\beta$ 保证下降，不满足它也未必发散（有些方向可能仍然收敛）。反过来，满足它也**不保证**每步都下降，因为它只约束了一阶近似，实际问题里还有浮点误差、小批量噪声、非光滑点。第 9 节的感知机就是活生生的例子：它的“梯度”是次梯度，下降引理根本不适用，实测有超过一半的更新让总损失上升。

4.4 从“每步下降”到“收敛”

下降引理只说“每步降一点”，降一点能不能保证最终到达谷底？还需要防止一种情况：步子越来越小，停在半山腰。对凸函数可以排除这种情况。

把引理在 $\eta = 1/\beta$ 下写成

$$L(\mathbf{w}_{k+1}) \le L(\mathbf{w}_k) - \frac{1}{2\beta}\|\nabla L(\mathbf{w}_k)\|^2$$

移项并对 $k = 0, \dots, K-1$ 求和，中间项抵消：

$$\sum_{k=0}^{K-1} \|\nabla L(\mathbf{w}_k)\|^2 \;\le\; 2\beta\big[L(\mathbf{w}_0) - L(\mathbf{w}_K)\big] \;\le\; 2\beta\big[L(\mathbf{w}_0) - L^\star\big]$$

右边是常数，所以左边这个和有限，梯度的平方平均必然趋于零：

$$\min_{0\le k<K} \|\nabla L(\mathbf{w}_k)\|^2 \;\le\; \frac{2\beta\,[L(\mathbf{w}_0) - L^\star]}{K} \;\longrightarrow\; 0$$

这就是梯度下降的收敛性：**梯度的最小值以 $O(1/K)$ 的速度趋于零**。若 L 还是凸的，梯度为零意味着全局最优，于是 $L(\mathbf{w}_K) - L^\star = O(1/K)$。

这个结论的边界，比结论本身更重要

- 它保证的是“梯度趋于零”，**不是**“到达全局最优”。非凸时梯度为零的点可能是鞍点或局部极小。
- 它保证的是“某个 k 很好”，**不是**“最后一个 k 很好”，实际中常需要记录历史最优点。
- $O(1/K)$ 是对最坏情况的上界，实践中常常更快（尤其强凸时是指数收敛）。

5 二次函数的完整解

选读

二次函数是唯一能写出**精确**收敛公式的情形。它值得单独看，因为它把“条件数”“振荡”“收敛速率”这些模糊的词变成了可计算的数。

5.1 坐标变换：解耦

考虑

$$L(\mathbf{w}) = \frac{1}{2}\mathbf{w}^\top \mathbf{A}\mathbf{w} - \mathbf{b}^\top \mathbf{w}, \qquad \mathbf{A} = \mathbf{A}^\top \succ 0$$

梯度 $\nabla L = \mathbf{A}\mathbf{w} - \mathbf{b}$，最优解满足 $\mathbf{A}\mathbf{w}^\star = \mathbf{b}$。更新式两边减去 $\mathbf{w}^\star$：

$$\mathbf{w}_{k+1} - \mathbf{w}^\star = \mathbf{w}_k - \eta(\mathbf{A}\mathbf{w}_k - \mathbf{b}) - \mathbf{w}^\star = (\mathbf{I} - \eta\mathbf{A})(\mathbf{w}_k - \mathbf{w}^\star)$$

记误差 $\mathbf{e}_k = \mathbf{w}_k - \mathbf{w}^\star$，则 $\mathbf{e}_{k+1} = (\mathbf{I} - \eta\mathbf{A})\mathbf{e}_k$，于是

$$\mathbf{e}_K = (\mathbf{I} - \eta\mathbf{A})^K \mathbf{e}_0$$

矩阵的幂不好直接看，但 $\mathbf{A}$ 对称正定，可以正交对角化 $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$，其中 $\mathbf{Q}$ 的列是单位正交特征向量，$\mathbf{\Lambda} = \mathrm{diag}(\lambda_1, \dots, \lambda_d)$。令 $\mathbf{z} = \mathbf{Q}^\top \mathbf{e}$（相当于把坐标旋转到特征向量方向），则

$$\mathbf{z}_{k+1} = (\mathbf{I} - \eta\mathbf{\Lambda})\mathbf{z}_k \quad \Longrightarrow \quad z_k^{(i)} = (1 - \eta\lambda_i)^k \, z_0^{(i)}$$

在一个“好”的坐标系里，d 维问题拆成了 d 个互不干扰的一维问题，每个方向独立地按自己的比率 $(1 - \eta\lambda_i)$ 衰减。这是整篇文章最漂亮的一步。

5.2 三个结论

（1）收敛条件。所有方向都要衰减，需要 $|1 - \eta\lambda_i| < 1$ 对所有 i 成立，即

$$0 < \eta < \frac{2}{\lambda_{\max}}$$

与第 4 节的 $\eta < 2/\beta$ 一致，因为 $\beta = \lambda_{\max}$。

（2）收敛速率由条件数决定。最慢的方向是 λ 最小（或最大）的那个，其衰减率是 $\max_i |1 - \eta\lambda_i|$。可以证明最优步长 $\eta^\star = \frac{2}{\lambda_{\max} + \lambda_{\min}}$，此时最慢衰减率为

$$\frac{\kappa-1}{\kappa+1}, \qquad \kappa=\frac{\lambda_{\max}}{\lambda_{\min}}$$

κ 叫条件数。$\kappa=1$（各方向曲率相同）时衰减率为 0，一步到位；$\kappa=100$ 时衰减率约 0.98，要几百步才降一个数量级。**这就是“狭长山谷里梯度下降很慢”的定量版本。**

（3）振荡的根源。若某个方向的 $\eta\lambda_i>1$，则 $1-\eta\lambda_i<0$，该方向每步变号，表现为来回横跳。曲率大的方向最容易振荡，因为 λ_i 大。实际训练中看到的“之字形”，就是不同方向衰减率差异过大的直接结果。

算一遍

取 $L=\frac{1}{2}(w_1^2+10w_2^2)$，从 $(10,1)$ 出发，$\lambda=(1,10)$，$\beta=10$，稳定条件 $\eta<0.2$。

- $\eta=0.1$：两方向衰减率 0.9 与 0。w_1 每步乘 0.9（慢），w_2 一步到位（快）。所以轨迹几乎是一条沿 w_1 轴缓慢滑行的直线。
- $\eta=0.15$：衰减率 0.85 与 -0.5。w_2 仍然快，但每步变号，轨迹呈小幅锯齿。
- $\eta=0.19$：衰减率 0.81 与 -0.9。w_2 方向几乎不衰减地来回跳，收敛极慢，这就是“贴着稳定边界走”的代价。

三个步长都满足 $\eta<0.2$，都没有发散，但收敛速度差出一个数量级。**“不发散”和“收敛得快”是两件独立的事。**

6 三种梯度下降

到这里，公式 $\mathbf{w}\leftarrow\mathbf{w}-\eta\nabla L(\mathbf{w})$ 里的梯度，指的是**在整个训练集上算出的**梯度。但真实数据可能有几百万条，每算一次梯度都要把所有样本过一遍，代价极高。于是有了三种变体。

6.1 批量梯度下降（BGD）

$$\nabla L(\mathbf{w})=\frac{1}{N}\sum_{i=1}^{N}\nabla\ell\big(\mathbf{w};\mathbf{x}_i,y_i\big)$$

每次用全部 N 个样本算梯度。优点是方向准、下降稳定；缺点是单步代价 $O(N)$，数据大时一步都走不动。第 2-5 节分析的都是它。

6.2 随机梯度下降（SGD）

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell\big(\mathbf{w}; \mathbf{x}_{i_k}, y_{i_k}\big)$$

每次只取**一个**样本 i_k（通常按随机顺序轮换），用它的梯度近似整体梯度。单步代价 $O(1)$，同样时间内能更新 N 次。

关键在于：如果样本是均匀随机抽取的，那么单样本梯度是整体梯度的**无偏估计**：

$$\mathbb{E}_i\big[\nabla \ell(\mathbf{w}; \mathbf{x}_i, y_i)\big] = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell(\mathbf{w}; \mathbf{x}_i, y_i) = \nabla L(\mathbf{w})$$

所以“平均而言”它指向正确的方向，但每一步都有噪声，方差约为 σ^2。这带来两个后果：

- **好的一面**：噪声可以帮助跳出较浅的局部极小和鞍点，也带来正则化效果；
- **坏的一面**：即使到了最优点附近，梯度也不为零，参数会在最优点周围**不停抖动**，无法精确停住。因此 SGD 必须配合**递减的学习率**（如 $\eta_k \propto 1/\sqrt{k}$ 或 $1/k$），才能保证收敛。

6.3 小批量（mini-batch）

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{B}\sum_{i \in \mathcal{B}_k} \nabla \ell(\mathbf{w}; \mathbf{x}_i, y_i)$$

每次取 B 个样本（如 32、128、256）。这是实践中的默认选择，原因是它同时利用了两件事：

- **矩阵运算的效率**：B 个样本的梯度可以用一次矩阵乘法算完，GPU 上 $B = 128$ 的耗时和 $B = 1$ 相差无几；
- **方差与代价的折中**：B 个独立样本平均后，梯度方差降到 σ^2/B，方向更稳。

方法	单步用的样本	梯度方差	单步代价	典型用途
BGD	全部 N	0	$O(Nd)$	小数据、凸问题、理论分析
SGD	1	σ^2	$O(d)$	在线学习、超大样本流
mini-batch	B	σ^2/B	$O(Bd)$	深度学习默认

> **收敛速度的差别**
>
> 凸且光滑时，BGD 的损失以 $O(1/K)$ 下降（K 是**迭代次数**）；SGD 由于噪声，只能达到 $O(1/\sqrt{K})$。但一次迭代的代价相差 N 倍，所以按**总计算量**比较时，SGD

> 往往更快。这是一个“迭代次数”与“单次代价”的经典权衡，不能只看其中一个。

7 手算全过程

理论说完了，现在把它跑一遍。这个例子小到可以在纸上算完，但已经包含了多维梯度下降的所有特征。

目标函数

$$L(w_1, w_2) = \frac{1}{2}\left(w_1^2 + 3w_2^2\right)$$

梯度

$$\nabla L = \begin{pmatrix} w_1 \\ 3w_2 \end{pmatrix}$$

初始点 $(w_1, w_2) = (2,\, 1)$，学习率 $\eta = 0.5$。

先检查稳定性：$\mathbf{A} = \mathrm{diag}(1, 3)$，$\lambda_{\max} = 3$，要求 $\eta < 2/3 \approx 0.667$，0.5 满足。而且第 5 节给出最优步长 $\eta^\star = 2/(\lambda_{\max} + \lambda_{\min}) = 2/4 = 0.5$，**正好取到了最优值**。

每一步的更新

$$\begin{pmatrix} w_1 \\ w_2 \end{pmatrix}_{k+1} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}_{k} - 0.5 \begin{pmatrix} w_1 \\ 3w_2 \end{pmatrix}_{k} = \begin{pmatrix} 0.5\, w_1 \\ -0.5\, w_2 \end{pmatrix}_{k}$$

k	w_1	w_2	$\partial L/\partial w_1 = w_1$	$\partial L/\partial w_2 = 3w_2$	L
0	2	1	2	3	3.5
1	1	−0.5	1	−1.5	0.875
2	0.5	0.25	0.5	0.75	0.21875
3	0.25	−0.125	0.25	−0.375	0.054688
4	0.125	0.0625	0.125	0.1875	0.013672
5	0.0625	−0.03125	0.0625	−0.09375	0.003418

最优解是 $(0, 0)$，最优值 $L^\star = 0$。第 0 行是初值，梯度在更新之前计算。

读这张表，能看出四件事：

1. w_1 **严格单调减半**：$2 \to 1 \to 0.5 \to 0.25 \cdots$，因为 $1 - \eta\lambda_1 = 1 - 0.5 = 0.5$。

2. w_2 **每步变号并且减半**：$1 \to -0.5 \to 0.25 \to -0.125 \cdots$，因为 $1 - \eta\lambda_2 = 1 - 1.5 = -0.5$。负号就是“跨过谷底”的直接证据。
3. L **每步恰好缩小到四分之一**：因为两个方向都乘了 0.5，二次型按 $0.5^2 = 0.25$ 缩放。这正好验证第 5 节的衰减率 $(\kappa - 1)/(\kappa + 1) = 2/4 = 0.5$，以及 $\kappa = 3$。
4. **梯度的大小随** L **同步缩小**：越接近谷底，步子自动越小。这是梯度下降“自动减速”的机制，也是它不需要手动调整步长衰减的原因（凸二次情形）。

> **注意第 2 点**
>
> w_2 在 1 和 -0.5 之间来回跳，看起来像是“错了”，其实完全正确：w_2 的曲率是 3，最优步长的两倍正好等于它的周期，所以每步都精确落在对面。这种“振荡但收敛”是梯度下降的正常行为，不要误判为发散。判断标准只有一个：**误差的绝对值是不是在缩小**。

8 收敛性到底保证了什么

“梯度下降会收敛”这句话，在不同假设下含义完全不同。这一节把三种情形分开说清楚，因为混用它们是最常见的理论误读。

8.1 凸且光滑：$O(1/K)$

若 L 凸、β-光滑，取 $\eta = 1/\beta$，则

$$L(\mathbf{w}_K) - L^\star \;\le\; \frac{\beta\|\mathbf{w}_0 - \mathbf{w}^\star\|^2}{2K} = O\!\left(\frac{1}{K}\right)$$

含义：迭代次数增加 10 倍，误差降到约 $1/10$。想再降一个数量级，就要再花 10 倍步数。**这是“次线性”收敛**，慢，但对一般凸问题已经是最优的一阶方法速率（有下界证明）。

8.2 强凸且光滑：线性收敛

若还满足 μ-强凸（曲率有下界 $\mu > 0$），则

$$L(\mathbf{w}_K) - L^\star \;\le\; \left(1 - \frac{\mu}{\beta}\right)^{K} \big(L(\mathbf{w}_0) - L^\star\big)$$

误差按固定比例缩小，叫**线性收敛**（在数值分析里“线性”指误差等比，不是指函数是线性的）。速率由条件数 $\kappa = \beta/\mu$ 决定：κ 越大越慢。第 5 节的二次函数正是这种情形，那时 $\mu = \lambda_{\min}$、$\beta = \lambda_{\max}$，与 $(\kappa-1)/(\kappa+1)$ 的结论一致（量级相同）。

8.3 非凸：只能到驻点

神经网络的损失几乎都是非凸的。此时能保证的只有 4.4 节那条：

$$\min_{k<K} \|\nabla L(\mathbf{w}_k)\|^2 \;\le\; O\!\left(\frac{1}{K}\right)$$

即**存在某一步梯度很小**。而“梯度小”可能是极小点，也可能是鞍点，还可能是平坦的高原。理论上无法区分，实践中靠随机初始化、噪声、动量等手段逃离。

> **”收敛”不等于”找到最优”**
>
> 非凸时，梯度下降甚至可能收敛到一个**比初始点还差的局部极小**。这不是实现 bug，而是问题本身的性质。深度学习中它之所以不致命，是因为高维空间的局部极小往往接近全局最优，真正麻烦的是鞍点和大片平坦区域。

8.4 一个特殊情形：感知机

你在学感知机，它正好是一个“看起来不该收敛、却能被证明收敛”的例子。它的损失

$$L(\mathbf{w}, b) = -\sum_{i \in M} y_i \left(\mathbf{w}^\top \mathbf{x}_i + b\right)$$

是**分段线性**的（不是光滑的），而且 M 随参数变化，所以第 4 节的下降引理不适用，实测中确实会出现“某次更新让总损失上升”。

但它有另一个结论，来自 Novikoff 定理：若数据线性可分，且 $\|\hat{\mathbf{x}}_i\| \le R$、存在单位向量 $\mathbf{u}$ 使 $y_i \mathbf{u}^\top \hat{\mathbf{x}}_i \ge \gamma$，则误分类次数

$$k \;\le\; \left(\frac{R}{\gamma}\right)^2$$

这里 $\hat{\mathbf{x}} = (\mathbf{x}, 1)$ 是把偏置吸收进去的增广向量。**它保证的是“有限步停止”，不是“损失单调下降”**。两种收敛性（函数值下降 vs 迭代终止）在非光滑问题里会分家，这是很好的提醒：不要用一个定理的结论去套另一个问题的假设。

9 三个真实模型的推导

梯度下降的公式是通用的，但每个模型的梯度长得不一样。这一节把三个最基础的模型完整推一遍，你会发现它们最后都落到同一个更新形式上。

9.1 线性回归（平方损失）

模型：$\hat{y} = \mathbf{w}^\top \mathbf{x}$（为简洁省略截距，把它并进 $\mathbf{x}$ 即可）。

损失（均方误差的一半，N 个样本）：

$$L(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^{N} \left(\mathbf{w}^\top \mathbf{x}_i - y_i \right)^2$$

求梯度：记残差 $r_i = \mathbf{w}^\top \mathbf{x}_i - y_i$。对单个样本，用链式法则：

$$\frac{\partial}{\partial \mathbf{w}} \frac{1}{2} r_i^2 = r_i \cdot \frac{\partial r_i}{\partial \mathbf{w}} = r_i \, \mathbf{x}_i$$

（这里用到 $\partial(\mathbf{w}^\top \mathbf{x}_i)/\partial \mathbf{w} = \mathbf{x}_i$，可以逐分量验证：$\partial(\sum_j w_j x_{ij})/\partial w_k = x_{ik}$。）对所有样本求和：

$$\nabla L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} r_i \mathbf{x}_i = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times d}$ 每行是一个样本。**更新**：

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

> **一个容易忽略的点**
>
> 对线性回归，令梯度为零可以直接解出 $\mathbf{w}^\star = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$，这就是正规方程。既然有解析解，为什么还用梯度下降？因为 d 很大时求逆是 $O(d^3)$、存 $\mathbf{X}^\top \mathbf{X}$ 要 $O(d^2)$ 内存，而梯度下降每步只要 $O(Nd)$、内存 $O(d)$。**算法选择取决于规模，不是取决于哪个“更高级”。**

9.2 逻辑回归（交叉熵）

模型：$p=\sigma(z)$，其中 $z=\mathbf{w}^\top\mathbf{x}$，$\sigma(z)=\dfrac{1}{1+e^{-z}}$ 是 sigmoid。

损失（负对数似然，标签 $y\in\{0,1\}$）：

$$L(\mathbf{w})=-\frac{1}{N}\sum_{i=1}^{N}\Big[y_i\log\sigma(z_i)+(1-y_i)\log\big(1-\sigma(z_i)\big)\Big]$$

关键性质：sigmoid 的导数有一个漂亮的形式，先算出来：

$$\sigma'(z)=\frac{e^{-z}}{(1+e^{-z})^2}=\sigma(z)\big(1-\sigma(z)\big)$$

对单个样本求导，记 $p_i=\sigma(z_i)$：

$$\frac{\partial\ell_i}{\partial z_i}=-\frac{y_i}{p_i}\cdot p_i(1-p_i)+\frac{1-y_i}{1-p_i}\cdot p_i(1-p_i)=-y_i(1-p_i)+(1-y_i)p_i=p_i-y_i$$

再乘上 $\partial z_i/\partial\mathbf{w}=\mathbf{x}_i$：

$$\nabla L(\mathbf{w})=\frac{1}{N}\sum_{i=1}^{N}(p_i-y_i)\,\mathbf{x}_i=\frac{1}{N}\mathbf{X}^\top(\mathbf{p}-\mathbf{y})$$

注意它的形状与线性回归**完全一致**：都是“残差 × 特征”再平均。区别只在残差的定义：线性回归用 $\hat{y}-y$，逻辑回归用 $p-y$。这不是巧合，而是广义线性模型的结构。

9.3 感知机（合页型损失）

模型：$f(\mathbf{x})=\operatorname{sign}(\mathbf{w}^\top\mathbf{x}+b)$，标签 $y\in\{-1,+1\}$。

损失：只统计被分错的点，

$$L(\mathbf{w},b)=-\sum_{i\in M}y_i\left(\mathbf{w}^\top\mathbf{x}_i+b\right),\qquad M=\{i:\ y_i(\mathbf{w}^\top\mathbf{x}_i+b)\le 0\}$$

求梯度：对 M 内每个点，$y_i(\mathbf{w}^\top\mathbf{x}_i+b)$ 对 $\mathbf{w}$ 的导数是 $y_i\mathbf{x}_i$，前面有负号：

$$\nabla_{\mathbf{w}}L=-\sum_{i\in M}y_i\mathbf{x}_i,\qquad\frac{\partial L}{\partial b}=-\sum_{i\in M}y_i$$

更新：减梯度等于加

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_{i \in M} y_i \mathbf{x}_i, \qquad b \leftarrow b + \eta \sum_{i \in M} y_i$$

教材里常见的写法是每次只取一个误分类点（随机梯度）：

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i \mathbf{x}_i, \qquad b \leftarrow b + \eta\, y_i$$

> **为什么这里不能套用第 4 节的定理**
>
> L 在 $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 0$ 处有折点，不可微；梯度只在严格误分类（$y_i f_i < 0$）时存在。此时更新方向是**次梯度**，下降引理不成立，“每步都下降”是错的。你在上一轮看到的统计——28 次更新中有 15 次总损失上升——正是这个原因。

9.4 三者对照

模型	损失	梯度	更新
线性回归	$\frac{1}{2N}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|^2$	$\frac{1}{N}\mathbf{X}^\top(\mathbf{X}\mathbf{w}-\mathbf{y})$	$\mathbf{w} - \eta\cdot$ 梯度
逻辑回归	交叉熵	$\frac{1}{N}\mathbf{X}^\top(\mathbf{p}-\mathbf{y})$	$\mathbf{w} - \eta\cdot$ 梯度
感知机	$-\sum_M y_i f_i$	$-\sum_M y_i \mathbf{x}_i$	$\mathbf{w} + \eta \sum_M y_i \mathbf{x}_i$

前两行是“残差 × 特征”，第三行因为损失自带负号，更新变成加法。三者都是同一个公式的特例。

10 八个常见误区

这些说法在教材和博客里反复出现，但每一句都需要加上限定条件才成立。

误区 1：梯度下降每一步都会让损失变小

只在可微、$\eta < 2/\beta$、且用真实梯度的条件下成立。小批量有噪声、感知机不可微、步长过大时都可能上升。**正确说法**：在满足条件的凸光滑问题上，损失单调不增。

误区 2：负梯度是“下降最快的方向”

只在**无穷小步长 + 欧氏长度**下成立。第 3.3 节的数值表里，夹角 89° 的方向一阶预测还有微小下降，实测却上升了。有限步长时，二阶项可以改变排序。

误区 3：梯度为零就是最小值

可能是极大值、鞍点，或者平坦区域。一维反例 $f(w) = w^3$ 在 $w = 0$ 处梯度为零，但那既不是极大也不是极小。高维中鞍点远比局部极小常见。

误区 4：学习率越小越好

太小会导致收敛极慢、在平坦区停滞，还可能因浮点精度而“假收敛”（更新量小于机器精度）。正确做法是选在稳定上界内、配合衰减策略。

误区 5：损失下降就说明模型变好了

训练损失下降只说明**拟合训练集**更好，可能是过拟合。要同时看验证集。梯度下降优化的是训练损失，不直接优化泛化能力。

误区 6：特征不用标准化

标准化会改变 β 和 μ，从而改变稳定步长和条件数。某列量纲是万元、另一列是比例时，梯度会被大尺度特征主导，收敛极慢甚至发散。**标准化不是可选的美化步骤，而是算法能正常工作的前提之一。**

误区 7：SGD 只是“更快的梯度下降”

它更快是因为单步便宜，但收敛速率更差（$O(1/\sqrt{K})$ 对 $O(1/K)$），而且不会精确停在最优点。它的价值在于**单位计算量下的进展**，以及噪声带来的逃离鞍点能力。

误区 8：感知机的学习率会影响收敛速度

不会。η 只把 $(\mathbf{w}, b)$ 整体缩放，分类判定只看符号，所以更新序列与 η 无关，迭代次数完全相同（实测 $\eta = 0.01, 1, 100$ 都是 8 次）。这是感知机损失**正齐次**的特殊后果，不能推广到别的模型。

11 自测

合上讲义，先自己算，再展开对照。

题 1

$f(w)=w^3$，从 $w_0=1$ 出发，$\eta=0.1$，沿负梯度走一步后 w_1 是多少？f 下降了吗？如果继续走，会停在哪里？

解答

$f'(w)=3w^2$，$f'(1)=3$，所以 $w_1=1-0.1\times 3=0.7$。$f(1)=1$，$f(0.7)=0.343$，确实下降。

但 $w=0$ 处 $f'=0$，梯度下降会停在那里——而它不是极小值（左边更小）。这说明“梯度为零”只是**驻点**，不保证最优。这个例子还说明：非凸问题上，梯度下降的结果强烈依赖初始点。

题 2

$L(w)=2w^2$，从 $w_0=1$ 出发。分别取 $\eta=0.1$、0.2、0.3、0.6，写出 w_k 的通项，判断是否收敛。

解答

这里 $\lambda=L''=4$，$w^\star=0$，误差递推 $e_{k+1}=(1-4\eta)e_k$。

- $\eta=0.1$：$1-0.4=0.6$，$w_k=0.6^k$，单调收敛。
- $\eta=0.2$：$1-0.8=0.2$，$w_k=0.2^k$，收敛更快。
- $\eta=0.3$：$1-1.2=-0.2$，$w_k=(-0.2)^k$，收敛但每步变号。
- $\eta=0.6$：$1-2.4=-1.4$，$|-1.4|>1$，发散。

临界值 $2/\lambda=0.5$。注意 $\eta=0.3$ 仍收敛，只是振荡——**振荡不等于发散**，判断依据是 $|1-\eta\lambda|$ 是否小于 1。

题 3

$L = \frac{1}{2}(w_1^2 + 100 w_2^2)$，用梯度下降会有什么现象？如何改进（至少说两种）？

解答

条件数 $\kappa = 100$。为保证 w_2 方向不振荡，需要 $\eta < 2/100 = 0.02$；但此时 w_1 方向的衰减率是 $1 - 0.02 = 0.98$，每步只缩小 2%，要几百步才能收敛。这就是“一个方向被另一个方向绑住”的典型现象。

改进：①标准化特征，让两列尺度接近，κ 变小；②用牛顿法或拟牛顿法，方向改为 $-\mathbf{H}^{-1}\nabla L$；③用动量、Adam 等自适应方法，按方向调整步长；④坐标缩放/预条件。任答两种即可。

题 4

感知机的损失在 $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 0$ 处可微吗？这对“每步下降”的结论有什么影响？

解答

不可微。该点左侧（函数间隔为负）斜率是 $-y_i \mathbf{x}_i$，右侧（间隔为正，该点已不在 M 中）斜率是 0，左右导数不相等，是一个折点。

因此只能用**次梯度**（两侧斜率的凸组合）。次梯度法不保证每步下降，只保证在合适步长下“到最优解的距离不增”之类的弱结论；实测中确实会出现总损失上升。

题 5

用自己的话解释：为什么“沿负梯度走”能让所有参数同时下降，而别的方向不行？

解答

一阶变化可以按参数拆成 $\sum_j (\partial L/\partial w_j)\Delta w_j$。只有取 $\Delta w_j = -\eta\, \partial L/\partial w_j$ 时，每一项都等于 $-\eta(\partial L/\partial w_j)^2 \le 0$，没有一项为正。

换任何其他方向，总有些 Δw_j 与对应偏导同号，那些参数就在增加损失，需要靠别的参数补偿，净下降必然更少。这就是“无内耗”的含义，也是 Cauchy–Schwarz 不等式等号条件的直观版本。

12 代码附录

一个不依赖任何机器学习库的实现，把本文所有公式落到实处。它同时支持批量和小批量，并记录每步的损失，方便你自己验证第 7 节的表格。

```python
import numpy as np

class GradientDescent:
    """最小二乘 / 一般可微损失的梯度下降。"""

    def __init__(self, lr=0.1, epochs=100, batch_size=None, seed=0):
        self.lr = lr
        self.epochs = epochs
        self.batch_size = batch_size      # None 表示批量梯度下降
        self.seed = seed

    def fit(self, X, y):
        X, y = np.asarray(X, float), np.asarray(y, float)
        n, d = X.shape
        w = np.zeros(d)
        self.history_ = []
        rng = np.random.default_rng(self.seed)

        for _ in range(self.epochs):
            idx = rng.permutation(n) if self.batch_size else np.arange(n)
            bs = self.batch_size or n
            for s in range(0, n, bs):
                b = idx[s:s + bs]
                grad = X[b].T @ (X[b] @ w - y[b]) / len(b)   # 第 9.1 节
                w -= self.lr * grad
                self.history_.append(0.5 * np.mean((X @ w - y) ** 2))
        self.w_ = w
        return self

    def predict(self, X):
        return np.asarray(X, float) @ self.w_


if __name__ == "__main__":
    # 复现第 7 节的二次函数：L = 0.5*(w1^2 + 3*w2^2)
    A = np.diag([1.0, 3.0])
    w = np.array([2.0, 1.0])
    eta = 0.5
    for k in range(6):
        L = 0.5 * w @ A @ w
        print(f"k={k}  w=({w[0]:.4f}, {w[1]:.4f})  L={L:.6f}")
        w = w - eta * (A @ w)
```

运行输出应当与第 7 节的表格逐行吻合：

```
k=0  w=(2.0000, 1.0000)  L=3.500000
k=1  w=(1.0000, -0.5000)  L=0.875000
k=2  w=(0.5000, 0.2500)  L=0.218750
k=3  w=(0.2500, -0.1250)  L=0.054688
k=4  w=(0.1250, 0.0625)  L=0.013672
k=5  w=(0.0625, -0.0313)  L=0.003418
```

> **怎么用它检验自己的理解**
>
> 改三个地方试：把 `eta` 改成 0.7（超过 2/3）看何时发散；把 `A` 改成 `diag([1, 100])` 看两个方向的收敛速度差异；把 L 改成 w^3 看它停在 $w=0$。这三个实验对应第 5、8、10 节的全部结论。

结语：这份推导的骨架

把全文压缩成五行，就是梯度下降的全部逻辑：

1. **目标**：找一个 $\mathbf{w}$ 使 $L(\mathbf{w})$ 最小。除了极少数情形，没有解析解。
2. **信息**：只有当前点的梯度 ∇L，它给出每个参数的一阶敏感度。
3. **方向**：$-\nabla L$ 是唯一让所有参数同向下坡的方向（Cauchy-Schwarz 等号条件）。
4. **步长**：必须满足 $\eta < 2/\beta$，否则一阶近似失效、误差被放大。
5. **保证**：凸时损失以 $O(1/K)$ 下降；非凸时只能到驻点；不可微时连“每步下降”都没有。

再往深处走，有三个自然的岔路：**二阶方法**（用 Hessian 修正方向，解决条件数）、**自适应步长**（Adam、RMSProp，解决各方向尺度不一）、**随机优化理论**（噪声、方差缩减、非凸景观）。它们的出发点都是这份推导里某一条假设不够用。

如果你想把这条线接到感知机上，最值得自己推一遍的是**对偶形式**：把 $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ 代回，更新从“改 $\mathbf{w}$”变成“给 α_i 加 η”，整个过程只依赖 Gram 矩阵 $\mathbf{x}_i^\top \mathbf{x}_j$。这一步会解释为什么感知机能自然核化，也会让你看清“参数空间”和“样本空间”的对应关系。